

Student Registration System

Technical Documentation & Developer Handover Guide

Foundation module of a School Management System

Repository: `tebitramson69-netizen/Student_registration`

Author: Tebit Ramson Titih

Stack: PHP 8 · MySQL (MariaDB) · HTML5 · CSS3 · Apache (XAMPP)

Document version: 1.0

Date: 21 September 2026

Audience: Developers joining or reviewing this codebase

Table of Contents

1. Executive Summary

- What the system does today
- Scope clarification: "School Management System"

2. Technology Stack & Runtime Environment

3. Project Structure — File by File

4. System Architecture

- Architectural style
- Request lifecycle
- Navigation map

5. Database Design

- Reconstructed schema
- Recommended DDL
- Data integrity notes

6. Feature Walkthrough (Create, Read, Update, Delete, Export)

7. Front-End & Styling

8. Installation & Local Setup (XAMPP)

9. Security Review

10. Known Bugs & Root-Cause Analysis

11. Continuous Integration — Current State

12. Improvement Roadmap

13. Appendix A — Query Reference

14. Appendix B — Glossary

1. Executive Summary

1.1 What the system does today

This project is a **server-rendered PHP web application** that manages a register of students stored in a MySQL database. It implements a complete CRUD cycle plus reporting:

- **Create** — a registration form captures first name, last name, email and telephone.
- **Read** — a paginated, sortable, searchable table lists all registered students.
- **Update** — an edit form pre-filled with an existing student's record.
- **Delete** — a link-triggered deletion with a JavaScript confirmation prompt.
- **Export** — the full register is streamed to the browser as a downloadable CSV file.

There is no framework, no ORM, no build step and no JavaScript bundle. Every page is a standalone PHP script that connects to the database, executes SQL, and prints HTML. This is deliberately simple and is appropriate for the project's current size; it is also the main constraint discussed in sections 9 and 12.

1.2 Scope clarification: "School Management System"

Important for any developer reading this. The repository is frequently described as a "school management system". As the code currently stands it is precisely one module of such a system: the *student registry*. There is no authentication, no user roles, no classes, no courses, no teachers, no enrolment, no grades, no attendance and no fees. Nothing in this document should be read as implying those exist. Section 12 sets out the path from the current registry to a genuine school management system.

Stating this plainly matters: the codebase is 6 PHP files and roughly 300 lines. Anyone told to expect a full management platform will waste time looking for tables and modules that were never written.

2. Technology Stack & Runtime Environment

Layer	Technology	Notes
Language	PHP 8.x	Procedural style; no Composer dependencies, no autoloader, no <code>composer.json</code> .
Database	MySQL / MariaDB	Accessed through the <code>mysqli</code> extension, mixing the object-oriented constructor with procedural query functions.
Web server	Apache	Supplied by XAMPP in development. No <code>.htaccess</code> , no URL rewriting — URLs map directly to files.
Markup / styling	HTML5, CSS3	One shared stylesheet, <code>style.css</code> , plus an inline <code><style></code> block inside <code>table.php</code> .
Client scripting	Inline JavaScript only	A single <code>onclick="return confirm(...)"</code> on the delete link. No libraries.
Version control	Git / GitHub	Default branch <code>main</code> .
CI	GitHub Actions	A Laravel workflow that does not match this project — see section 11.

Connection parameters (development defaults)

Defined in `connection.php` and hard-coded:

```
$servername = "localhost";
$username   = "root";
$password   = "";           // empty – XAMPP default
$database   = "student_db";
```

3. Project Structure — File by File

Student_registration/	
├─ connection.php	Database connection; included by every other PHP file
├─ form.php	Registration form + INSERT handler (CREATE)
├─ table.php	Student list: search, sort, pagination (READ)
├─ update.php	Edit form + UPDATE handler (UPDATE)
├─ delete.php	DELETE handler (no view of its own) (DELETE)
├─ export.php	Streams the whole register as CSV (EXPORT)
├─ style.css	Shared stylesheet
├─ README.md	Short project description + setup notes
├─ docs/	This documentation
├─ .github/workflows/	
└─ laravel.yml	GitHub Actions workflow (misconfigured – section 11)

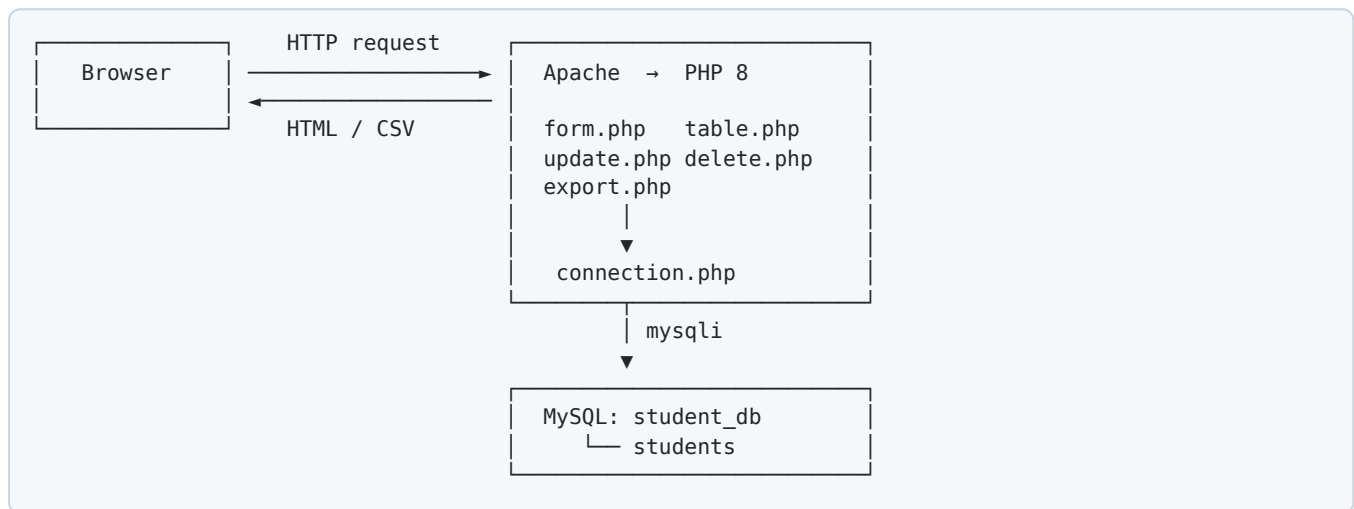
File	Lines	Responsibility
connection.php	12	Creates a <code>mysqli</code> object named <code>\$conn</code> and terminates the request with <code>die()</code> if the connection fails. Every page begins with <code>include "connection.php";</code> .
form.php	51	Dual-purpose: on <code>POST</code> it escapes the four fields and runs an <code>INSERT</code> , then redirects to <code>table.php</code> ; otherwise it renders the registration form. This is the de facto entry point of the application — there is no <code>index.php</code> .
table.php	137	The largest and most complex file. Handles pagination arithmetic, a whitelisted sort column and direction, an optional <code>LIKE</code> search across four columns, a total-row count, and renders the table with per-row Update/Delete links. Contains one helper function, <code>sortLink()</code> .
update.php	58	On <code>GET ?id=</code> it loads one student into <code>\$student</code> ; on <code>POST</code> it runs the <code>UPDATE</code> and redirects. Renders the pre-filled edit form.
delete.php	15	Reads <code>?id=</code> , casts it with <code>intval()</code> , deletes the row and redirects to <code>table.php</code> . Produces no HTML.
export.php	25	Sends CSV headers, opens <code>php://output</code> as a stream, writes a header row, then one <code>fputcsv()</code> per record.
style.css	156	Blue/white theme (<code>#007bff</code>), card-style form container, striped hoverable table, pagination buttons, focus states.

4. System Architecture

4.1 Architectural style

The application follows the classic **page-controller** pattern of traditional PHP: each URL is a physical file, and that file contains its own controller logic, data access and presentation. There is no routing layer, no separation of concerns, and no shared layout or template.

The consequence, which a new developer should understand immediately: **SQL, business rules and HTML live in the same file**. In `table.php`, lines 1–54 are PHP logic and lines 55–137 are the view. The two halves are separated only by the closing `?>` tag.



4.2 Request lifecycle

Every request follows the same five steps:

1. Apache maps the URL to a `.php` file and hands it to the PHP interpreter.
2. The script includes `connection.php`, which opens a fresh MySQL connection. *Each request opens its own connection; nothing is pooled or reused.*
3. The script inspects `$_SERVER['REQUEST_METHOD']`, `$_GET` and `$_POST` to decide what to do.
4. It builds an SQL string, executes it through `mysqli_query()`, and reads the result.
5. It either sends a `Location: redirect` (write operations) or echoes HTML (read operations).

Post/Redirect/Get is correctly applied. After a successful `INSERT`, `UPDATE` or `DELETE`, the code calls `header("Location: table.php"); exit();`. This prevents the browser from re-submitting the form on refresh and duplicating records. It is the strongest design decision in the codebase and should be preserved through any refactor.

4.3 Navigation map

URL	Method	Outcome
form.php	GET	Shows the empty registration form.
form.php	POST	Inserts the student, redirects to table.php .
table.php	GET	Lists students. Accepts ?page= , ?sort= , ?order= , ?search= .
update.php?id=N	GET	Shows the edit form filled with student N .
update.php	POST	Updates the student, redirects to table.php .
delete.php?id=N	GET	Deletes student N , redirects to table.php . See 9.2
export.php	GET	Downloads students.csv containing every record.

5. Database Design

5.1 Reconstructed schema

The repository contains **no .sql schema file and no migrations**. The structure below was reconstructed by reading every query in the codebase. Shipping an authoritative `database/schema.sql` is the single highest-value addition you can make to this repository — without it, no one can recreate the database.

Evidence used for the reconstruction:

- `form.php` inserts into `students(first_name, last_name, email, telephone)` — four user-supplied columns.
- `update.php` and `delete.php` filter on `id` after `intval()`, so `id` is an integer key.
- `table.php` whitelists exactly `['id','first_name','last_name','email','telephone']` as sortable columns — that is the full column list.
- `export.php` writes the header row `ID, First Name, Last Name, Email, Telephone` and then dumps `SELECT *` rows directly, which only lines up if the physical column order is `id, first_name, last_name, email, telephone`.

5.2 Recommended DDL

```
CREATE DATABASE IF NOT EXISTS student_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE student_db;

CREATE TABLE students (
  id          INT UNSIGNED NOT NULL AUTO_INCREMENT,
  first_name  VARCHAR(50)  NOT NULL,
  last_name   VARCHAR(50)  NOT NULL,
  email       VARCHAR(150) NOT NULL,
  telephone   VARCHAR(20)  NOT NULL,
  created_at  TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at  TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP
              ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  UNIQUE KEY uq_students_email (email),
  KEY idx_students_last_first (last_name, first_name)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

5.3 Data integrity notes

Concern	Analysis
Duplicate emails	Nothing in the PHP checks for an existing email before inserting. The same student can be registered any number of times. A <code>UNIQUE</code> constraint on <code>email</code> enforces this at the database level, which is the correct place — application checks alone are subject to race conditions.
Telephone as <code>VARCHAR</code>	Correct, and deliberately so. Cameroonian numbers are written with a leading <code>+237</code> and may carry spaces; an integer column would destroy both. Never store phone numbers as numeric types.
Character set	<code>connection.php</code> never calls <code>\$conn->set_charset('utf8mb4')</code> . Two consequences: accented characters common in Cameroonian names (é, ô, ç) may be mangled, and <code>mysqli_real_escape_string()</code> escapes according to the wrong charset, which weakens the project's only injection defence.
Indexing	Sorting is currently permitted on all five columns but only <code>id</code> is indexed. <code>ORDER BY last_name</code> over a large table causes a filesort. The composite index above covers the common alphabetical listing.
Search performance	The search uses <code>LIKE '%term%'</code> . A leading wildcard cannot use a B-tree index, so every search is a full table scan. Acceptable at a few thousand rows; at school scale, move to a <code>FULLTEXT</code> index.
Audit trail	There is no <code>created_at</code> / <code>updated_at</code> today, so it is impossible to know when a student was registered. The DDL above adds both.

6. Feature Walkthrough

6.1 Create — `form.php`

The file serves both the form and its handler. The method check at the top decides which:

```
if ($_SERVER['REQUEST_METHOD'] == "POST") {
    $first_name = mysqli_real_escape_string($conn, $_POST['first_name']);
    // ... last_name, email, telephone
    $sql = "INSERT INTO students(first_name, last_name, email, telephone)
        VALUES('$first_name', '$last_name', '$email', '$telephone')";
    if (mysqli_query($conn, $sql)) {
        header("Location: table.php");
        exit();
    } else {
        echo mysqli_error($conn);
    }
}
```

Validation is entirely client-side: `required` attributes and `type="email" / type="tel"`. Those are usability features, not security controls — any client that sends a raw POST bypasses them completely. There is no server-side validation at all.

Note also that the failure branch echoes `mysqli_error($conn)` straight to the page, leaking the raw database error (table names, column names, constraint names) to whoever triggered it. The same pattern appears in `update.php` and `delete.php`.

6.2 Read — `table.php`

Four concerns are handled in sequence:

Pagination

```
$limit = 15;
$page = isset($_GET['page']) && is_numeric($_GET['page']) ? (int)$_GET['page'] : 1;
$offset = ($page - 1) * $limit;
```

Sorting

This is handled correctly and is worth highlighting as good practice:

```
$validColumns = ['id', 'first_name', 'last_name', 'email', 'telephone'];
$sort = isset($_GET['sort']) && in_array($_GET['sort'], $validColumns) ? $_GET['sort'] :
    'first_name';
$order = isset($_GET['order']) && $_GET['order'] === 'DESC' ? 'DESC' : 'ASC';
```

Column and direction names cannot be parameterised by a prepared statement — they are part of the SQL grammar, not values. A whitelist is the correct defence, and the code applies it properly with a safe default. Keep this pattern when the rest of the file is migrated to prepared statements.

Search

If `?search=` is present and non-empty, the base query is replaced by a four-column `LIKE` query. The term is escaped with `mysqli_real_escape_string()`.

Rendering

Output is escaped with `htmlspecialchars()` on every cell and on the search box's echoed value, which blocks stored and reflected XSS on those paths. `sortLink()` builds the column header links and flips the direction when the same column is clicked twice.

6.3 Update — `update.php`

The `GET` branch loads the record; the `POST` branch writes it back, taking the row identifier from a hidden `<input name="id">`. Both branches use `intval()` on the id, which makes the `WHERE id = $id` clause safe. The four text fields use the same escaping as `form.php`. See section 10.3 for a real bug in the failure path of this file.

6.4 Delete — `delete.php`

```
if (isset($_GET['id'])) {
    $id = intval($_GET['id']);
    $sql = "DELETE FROM students WHERE id = $id";
    if (mysqli_query($conn, $sql)) { header("Location: table.php"); exit(); }
}
```

The `confirm()` dialog in `table.php` is a convenience for the user, nothing more. The deletion is a `GET` request, which is a significant design flaw — see section 9.2.

6.5 Export — `export.php`

The script sets `Content-Type: text/csv` and `Content-Disposition: attachment`, then streams rows through `php://output` rather than buffering the whole file in memory — the right approach for a growing dataset.

Hidden coupling. The header row is written by hand as `['ID', 'First Name', 'Last Name', 'Email', 'Telephone']`, but the data rows come from `SELECT *`. The two only line up while the physical column order matches. The moment someone adds a `created_at` column, every exported row silently gains a value with no matching header. Select the columns explicitly instead of using `*`.

7. Front-End & Styling

`style.css` defines a clean, consistent visual language: a light grey page background (`#f4f6f9`), a white card with a 12px radius and a soft shadow for forms, a primary blue (`#007bff`) for headings, buttons and table headers, zebra-striped rows, hover highlighting and focus rings on inputs. It is well written for its size.

Issues in the current front-end

Issue	Detail
Inline styles override the stylesheet	<code>table.php</code> links <code>style.css</code> and then declares an inline <code><style></code> block. Because it comes later, the inline rules win: the carefully styled blue table header and striped rows in <code>style.css</code> are replaced by a plain grey <code>#f4f4f4</code> header. The table page does not look like the rest of the app. Delete the inline block.
Invalid button markup	<code><button color="blue">...</button></code> nests a link inside a button, which is invalid HTML and behaves unpredictably. <code>color</code> is not a valid attribute. Use <code></code> and style <code>.btn</code> in CSS.
<code>update.php</code> is unstyled	It never links <code>style.css</code> , so the edit form renders as raw browser-default HTML while every other page is themed.
No responsive behaviour	<code>.container</code> is a fixed width: <code>400px</code> and the table has no horizontal scroll wrapper. On a phone — the dominant access device in Cameroon — the form overflows and the table is cut off. Use <code>max-width: 400px</code> and wrap the table in a container with <code>overflow-x: auto</code> .
Accessibility	<code><label for="..."></code> is present in <code>form.php</code> but the matching inputs have no <code>id</code> attribute, so the association never happens and screen readers cannot announce the fields. Add <code>id</code> to each input.

8. Installation & Local Setup (XAMPP)

1. **Install XAMPP** and start *Apache* and *MySQL* from the control panel.

2. **Clone into the web root:**

```
cd C:\xampp\htdocs
git clone https://github.com/tebitramson69-netizen/Student_registration.git
```

3. **Create the database.** Open `http://localhost/phpmyadmin`, create a database named `student_db`, and run the DDL from section 5.2 in the SQL tab.

4. **Check the credentials** in `connection.php`. The XAMPP defaults (`root` / empty password) already match.

5. **Open the application:** `http://localhost/Student_registration/form.php`
There is no `index.php`, so the directory root will show a file listing rather than the app.

Verification checklist: register a student → confirm the redirect lands on `table.php` with the new row visible → click a column header and confirm the sort direction flips → search for part of the name → click *Update*, change a field, save → click *Download CSV* and open the file. If all six work, the environment is correct.

9. Security Review

Findings are ordered by severity. Each states the root cause before the fix, because understanding why a class of bug exists prevents reintroducing it elsewhere.

9.1 **CRITICAL** No authentication or authorisation

Root cause: no session layer was ever built. Not one file calls `session_start()`.

Impact: anyone who can reach the URL can list every student, edit any record, delete any record, and download the complete register from `export.php` — full personal data of every student, including emails and phone numbers, with a single unauthenticated GET request. On a local XAMPP install this is contained; the moment it is deployed, it is a data breach.

Fix: add an `admin_users` table with `password_hash()`-stored credentials, a `login.php`, and an `auth.php` guard included at the top of every protected page that redirects to the login screen when `$_SESSION['user_id']` is absent. This is prerequisite work for anything else — build it before adding features.

9.2 **HIGH** Destructive action exposed over GET (CSRF)

Root cause: `delete.php` performs a state change in response to a GET request, and no CSRF token is verified anywhere in the project.

Impact: HTTP GET is defined as a safe, idempotent method; browsers, link prefetchers, chat previews and crawlers follow GET links freely. An `` embedded in any page an authenticated administrator visits deletes student 5 silently. The `confirm()` dialog offers no protection — it is client-side and is never executed in that scenario.

Fix: convert deletion to a `POST` form, and verify a per-session CSRF token on every state-changing request:

```
// on render
$_SESSION['csrf'] ??= bin2hex(random_bytes(32));
echo '<input type="hidden" name="csrf" value="' . $_SESSION['csrf'] . '">';

// on submit
if (!hash_equals($_SESSION['csrf'] ?? '', $_POST['csrf'] ?? '')) {
    http_response_code(403);
    exit('Invalid request.');
```

9.3 **HIGH** String-concatenated SQL instead of prepared statements

Root cause: every query is assembled by interpolating variables into a string. The defence relied upon is `mysqli_real_escape_string()`.

Current status: to be fair to the code, the escaping is applied consistently and `intval()` guards every id, so no *immediately exploitable* injection was found. But this is a fragile position, for three reasons:

- It depends on a developer remembering to escape *every* variable, *every* time. One omission is a full compromise.
- `mysqli_real_escape_string()` escapes according to the connection charset, and `connection.php` never sets one (section 5.3). Charset mismatches are a known route around this function.
- It offers no protection when a value lands outside quotes in the SQL string.

Fix: prepared statements everywhere. The rewrite of `form.php`:

```
$stmt = $conn->prepare(
    "INSERT INTO students (first_name, last_name, email, telephone)
    VALUES (?, ?, ?, ?)"
);
$stmt->bind_param("ssss", $first_name, $last_name, $email, $telephone);
$stmt->execute();
$stmt->close();
```

Values are then sent to MySQL separately from the SQL text and can never be parsed as SQL. Keep the `$validColumns` whitelist for `ORDER BY` — identifiers still cannot be bound.

9.4 MEDIUM No server-side validation

Root cause: validation exists only as HTML5 attributes.

Impact: a crafted POST stores an empty name, a 10,000-character string, or `"not-an-email"` in the email column. Data quality degrades silently.

Fix: validate on the server before touching the database — `trim()`, length bounds, `filter_var($email, FILTER_VALIDATE_EMAIL)`, and a telephone pattern appropriate to the target region (for Cameroon, `/^(\+237)?6[5-9][0-9]{7}$/` covers current mobile numbering). Re-render the form with the submitted values and the error messages rather than discarding the user's input.

9.5 MEDIUM Database errors printed to the browser

Root cause: `echo mysqli_error($conn)` in three files, plus `die("Connection failed: " . $conn->connect_error)` in `connection.php`.

Impact: schema details, file paths and server configuration are disclosed to an attacker, who gets a free map of the database.

Fix: log the real error with `error_log()` and show the user a neutral message. In production set `display_errors = Off` and `log_errors = On`.

9.6 MEDIUM CSV formula injection in `export.php`

Root cause: field values are written to the CSV without neutralising leading `=`, `+`, `-` or `@`.

Impact: a student registered with the name `=HYPERLINK("http://evil","Click")` is stored harmlessly, but when an administrator opens the export in Excel or LibreOffice the cell is evaluated as a formula. This is a stored attack that fires inside the victim's spreadsheet application, not in the browser.

Fix: prefix any value beginning with those characters with a single quote before writing it.

9.7 **LOW** Credentials committed to the repository

Root cause: `connection.php` hard-codes host, user, password and database name, and the file is tracked by Git.

Impact: harmless today (empty XAMPP default), but the pattern guarantees that a real production password will eventually be committed and published.

Fix: move the values into an untracked `config.php` (or environment variables), commit a `config.example.php` with placeholders, and add `config.php` to a `.gitignore` — the repository currently has none.

Security scorecard

Control	Status	Comment
Authentication	Absent	No session layer at all.
Authorisation / roles	Absent	No concept of a user.
CSRF protection	Absent	Deletes are GET requests.
SQL injection defence	Partial	Escaping + <code>intval()</code> ; no prepared statements.
XSS defence (output)	Good	<code>htmlspecialchars()</code> applied consistently in <code>table.php</code> and <code>update.php</code> .
<code>ORDER BY</code> whitelist	Good	Correct approach, safe default.
Post/Redirect/Get	Good	Applied after every write.
Server-side validation	Absent	Client-side only.
Error handling	Weak	Raw database errors are echoed.

10. Known Bugs & Root-Cause Analysis

10.1 **HIGH** Pagination is wrong whenever a search is active

Where: `table.php` lines 37–40.

```
$countResult = mysqli_query($conn, "SELECT COUNT(*) AS total FROM students");
$totalRows   = mysqli_fetch_assoc($countResult)['total'];
$totalPages  = ceil($totalRows / $limit);
```

Root cause: the count query is unconditional. It counts every row in the table, while the list query above it may have been narrowed by a `LIKE` filter. The two queries disagree about what "the result set" is.

Observable symptom: with 100 students, searching for a term that matches 3 of them still renders 7 page links. Pages 2 through 7 are empty.

Fix: build the `WHERE` clause once and apply it to both the `SELECT` and the `COUNT(*)`. Restructuring the file so the filter is a single variable used by both queries also removes the duplicated SQL.

10.2 **MEDIUM** A negative or zero page number breaks the listing

Where: `table.php` line 8.

Root cause: `is_numeric('-3')` returns `true`, so `$page` becomes `-3` and `$offset` becomes `-60`. MySQL rejects a negative `OFFSET`, `mysqli_query()` returns `false`, and `mysqli_fetch_all(false, ...)` then raises a fatal error. The same happens with `?page=0`.

Fix: clamp the value — `$page = max(1, (int)($_GET['page'] ?? 1));` — and, once `$totalPages` is known, clamp the upper bound too.

10.3 **MEDIUM** `update.php` crashes on a failed update or a missing id

Where: `update.php` lines 4–9 and 41–53.

Root cause: `$student` is only populated inside `if (isset($_GET['id']))`. The HTML below always runs and always dereferences `$student['id']`. Two paths reach it undefined:

- A `POST` whose `UPDATE` fails — the code echoes the error, does *not* exit, and falls through to the form. `$student` was never set, because there is no `$_GET['id']` on a `POST`.
- A `GET` with a non-existent id — `mysqli_fetch_assoc()` returns `null`, and on PHP 8.1+ `htmlspecialchars(null)` is deprecated; accessing `null['id']` is a warning that renders an empty form with no id.

Fix: after the fetch, guard explicitly — if no record was found, redirect back to `table.php` (or render a "student not found" message) and `exit()`. On the `POST` failure path, redirect back to `update.php?id=$id` with the error rather than falling through.

10.4 **LOW** Searching for "0" returns everything

Where: `table.php` line 22 — `!empty($_GET['search'])`.

Root cause: in PHP, the string `"0"` is falsy, so `empty("0")` is `true` and the search branch is skipped. This is one of PHP's classic type-juggling traps.

Fix: test the string explicitly: `if (isset($_GET['search']) && trim($_GET['search']) !== '')`.

10.5 **LOW** The export header row can drift from the data

Covered in section 6.5. Replace `SELECT *` with an explicit column list.

10.6 **LOW** Pagination renders every page as a link

Where: `table.php` lines 122–127.

Root cause: the loop runs from `1` to `$totalPages` with no window. At 3,000 students that is 200 links on every page load.

Fix: render a sliding window (first, last, and a few pages either side of the current one).

Fix priority

Order	Item	Reason
1	Add authentication (9.1)	Everything else is meaningless while the data is public.
2	POST + CSRF tokens for delete/update (9.2)	Prevents silent destructive requests.
3	Prepared statements across all files (9.3)	Removes a whole bug class permanently.
4	Fix search pagination (10.1)	Visible, daily-use functional bug.
5	Clamp page / guard <code>\$student</code> (10.2, 10.3)	Crashes on ordinary inputs.
6	Server-side validation (9.4)	Protects long-term data quality.
7	Commit <code>schema.sql</code> + <code>.gitignore</code> (5.1, 9.7)	Makes the project reproducible.
8	Front-end cleanup (section 7)	Consistency and mobile usability.

11. Continuous Integration — Current State

The repository contains `.github/workflows/laravel.yml`, added from GitHub's starter-workflow gallery. It does not correspond to this project:

Workflow step	Why it fails here
<code>copy('.env.example', '.env')</code>	Neither file exists in this repository.
<code>composer install</code>	There is no <code>composer.json</code> — the project has no dependencies.
<code>php artisan key:generate</code>	<code>artisan</code> is a Laravel console script. This is not a Laravel project.
<code>chmod -R 777 storage bootstrap/cache</code>	Neither directory exists.
<code>php artisan test</code>	There is no test suite of any kind.

Recommendation: replace it with a workflow that does something meaningful for a plain PHP project — a syntax check across every file, which catches parse errors before they are merged:

```
name: PHP Lint
on: [push, pull_request]
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: shivammathur/setup-php@v2
        with:
          php-version: '8.2'
      - name: Syntax check all PHP files
        run: find . -name "*.php" -print0 | xargs -0 -n1 php -l
```

Once validation logic is extracted into functions, add PHPUnit and test those functions directly. Testing is impossible while logic and HTML share a file — which is itself a strong argument for the refactor in section 12.

12. Improvement Roadmap

Three phases, ordered so that each one rests on the previous. Do not start phase 3 before phase 1 is complete.

Phase 1 — Harden what exists

- Sessions, login, logout, password hashing with `password_hash()` / `password_verify()`.
- An `auth.php` guard included at the top of every protected page.
- Prepared statements throughout; `set_charset('utf8mb4')` in the connection.
- CSRF tokens; delete converted to `POST`.

- Server-side validation with errors re-rendered into the form.
- `database/schema.sql`, `.gitignore`, and credentials moved out of version control.
- Fix bugs 10.1 – 10.4.

Phase 2 — Restructure for growth

- Add `index.php` as a proper entry point.
- Introduce `includes/header.php` and `includes/footer.php` so navigation and the stylesheet link live in one place instead of being duplicated per file.
- Move data access into functions (`getStudents()`, `findStudent()`, `createStudent()`, `updateStudent()`, `deleteStudent()`) in a `students.php` model file, so pages stop writing SQL directly. This single change is what makes the codebase testable.
- Switch to PDO, or at least settle on one `mysqli` style — the code currently mixes the object-oriented constructor with procedural query calls.
- Make the layout responsive; consolidate the two stylesheets into one.

Phase 3 — Become a school management system

Only at this point does the "school management" name become accurate. Suggested entity model:

```

students —< enrollments >— classes —< class_subjects >— subjects
      |                               |
      |                               |— teachers
      |—< attendance
      |—< grades      (student_id, subject_id, term, score)
      |—< fee_payments (student_id, term, amount, paid_at, method)

users (id, username, password_hash, role)   role ∈ {admin, teacher, secretary}

```

Build these one module at a time, each with its own CRUD, each behind the authorisation layer from phase 1. For a Cameroonian deployment, two things deserve early attention: bilingual (English/French) labels, and Mobile Money reconciliation in the fees module — both are far easier to design in from the start than to retrofit.

Appendix A — Query Reference

Every SQL statement in the codebase, in one place.

File	Statement
form.php	INSERT INTO students(first_name, last_name, email, telephone) VALUES(...)
table.php (list)	SELECT * FROM students ORDER BY {sort} {order} LIMIT {limit} OFFSET {offset}
table.php (search)	SELECT * FROM students WHERE first_name LIKE '%...%' OR last_name LIKE '%...%' OR email LIKE '%...%' OR telephone LIKE '%...%' ORDER BY {sort} {order} LIMIT ... OFFSET ...
table.php (count)	SELECT COUNT(*) AS total FROM students bug 10.1
update.php (load)	SELECT * FROM students WHERE id = {id}
update.php (save)	UPDATE students SET first_name=..., last_name=..., email=..., telephone=... WHERE id={id}
delete.php	DELETE FROM students WHERE id = {id}
export.php	SELECT * FROM students ORDER BY first_name ASC, last_name ASC

Appendix B — Glossary

Term	Meaning in this project
CRUD	Create, Read, Update, Delete — the four operations this application implements over the <code>students</code> table.
Post/Redirect/Get	Responding to a successful POST with a redirect, so refreshing the page does not resubmit the form. Used after every write here.
Prepared statement	An SQL statement sent to the database with placeholders, whose values are transmitted separately and therefore can never be interpreted as SQL. Not yet used in this project.
CSRF	Cross-Site Request Forgery — making a logged-in user's browser issue a state-changing request without their intent. <code>delete.php</code> is currently vulnerable.
XSS	Cross-Site Scripting — injecting script through stored or reflected data. Mitigated here by <code>htmlspecialchars()</code> on output.
Offset pagination	Fetching a page of rows with <code>LIMIT</code> and <code>OFFSET</code> . Simple, and what <code>table.php</code> uses.
Page controller	The architectural pattern where each URL maps to one file that handles logic and rendering together.
Formula injection	A stored value beginning with <code>=</code> , <code>+</code> , <code>-</code> or <code>@</code> that a spreadsheet program evaluates as a formula when the exported CSV is opened.

Document generated for the `Student_registration` repository. Every statement about the code was verified against the source at commit `4d11f42`. Where a file, table or feature does not exist, this document says so explicitly rather than describing an assumed implementation.